

Язык программирования С

Лекция 3: указатели на функции, виды памяти, стек, глобальные переменные, static

Лисица Никита

2026

“Универсальная” сортировка

- Хотим написать универсальную/обобщённую сортировку, работающую с любым типом данных: `int`, `float`, `const char*`, `struct product_t`

“Универсальная” сортировка

- Хотим написать универсальную/обобщённую сортировку, работающую с любым типом данных: `int`, `float`, `const char*`, `struct product_t`
- Проблемы:
 - Как передать массив объектов произвольного типа?

“Универсальная” сортировка

- Хотим написать универсальную/обобщённую сортировку, работающую с любым типом данных: `int`, `float`, `const char*`, `struct product_t`
- Проблемы:
 - Как передать массив объектов произвольного типа?
 - Как переставить (`swap`) местами объекты произвольного типа?

“Универсальная” сортировка

- Хотим написать универсальную/обобщённую сортировку, работающую с любым типом данных: `int`, `float`, `const char*`, `struct product_t`
- Проблемы:
 - Как передать массив объектов произвольного типа?
 - Как переставить (`swap`) местами объекты произвольного типа?
 - Как сравнить (`compare`) объекты произвольного типа?

- **void** – особый тип данных в C

- **void** – особый тип данных в C
- У него нет значений (функция, возвращающая **void**, ничего не возвращает)

- **void** – особый тип данных в C
- У него нет значений (функция, возвращающая **void**, ничего не возвращает)
- **void*** обычно используется как указатель на неизвестный тип (для него запрещена арифметика указателей)

- **void** – особый тип данных в C
- У него нет значений (функция, возвращающая **void**, ничего не возвращает)
- **void*** обычно используется как указатель на неизвестный тип (для него запрещена арифметика указателей)
- Любой тип указателя приводится к **void*** и обратно (в C++ для обратного преобразования нужно явное приведение типа)

- **void** – особый тип данных в C
- У него нет значений (функция, возвращающая **void**, ничего не возвращает)
- **void*** обычно используется как указатель на неизвестный тип (для него запрещена арифметика указателей)
- Любой тип указателя приводится к **void*** и обратно (в C++ для обратного преобразования нужно явное приведение типа)
- Некоторые функции (например, выделение памяти – **malloc**) возвращают **void***

- Мы можем использовать `void*`, чтобы принимать массив, который нам нужно отсортировать:

```
void sort(void *array, size_t count, ...);
```

Перестановка объектов (swap)

- Будем считать, что перестановка сводится к перестановке байт объектов

```
void sort(void *array, size_t count, size_t elem_size, ...) {
    char *ptr = array;

    ...
    swap(array + i * elem_size, array + j * elem_size, elem_size);
    ...
}

void swap(char *p1, char *p2, size_t elem_size) {
    char *end1 = p1 + elem_size;
    while (p1 != end1) {
        ...
    }
}
```

Сравнение объектов (compare)

- Для сравнения разных типов данных нужен разный код:

Сравнение объектов (compare)

- Для сравнения разных типов данных нужен разный код:
 - Сравнение `int` – оператор целочисленного сравнения

Сравнение объектов (compare)

- Для сравнения разных типов данных нужен разный код:
 - Сравнение `int` – оператор целочисленного сравнения
 - Сравнение `float` – оператор floating-point сравнения

Сравнение объектов (compare)

- Для сравнения разных типов данных нужен разный код:
 - Сравнение `int` – оператор целочисленного сравнения
 - Сравнение `float` – оператор floating-point сравнения
 - Сравнение строк (`const char*`) – лексикографическое сравнение циклом (`strcmp`)

Сравнение объектов (compare)

- Для сравнения разных типов данных нужен разный код:
 - Сравнение `int` – оператор целочисленного сравнения
 - Сравнение `float` – оператор floating-point сравнения
 - Сравнение строк (`const char*`) – лексикографическое сравнение циклом (`strcmp`)
 - Сравнение структур `struct product_t` по значению поля `weight` – доступ к полю + оператор сравнения

Сравнение объектов (compare)

- Для сравнения разных типов данных нужен разный код:
 - Сравнение `int` – оператор целочисленного сравнения
 - Сравнение `float` – оператор floating-point сравнения
 - Сравнение строк (`const char*`) – лексикографическое сравнение циклом (`strcmp`)
 - Сравнение структур `struct product_t` по значению поля `weight` – доступ к полю + оператор сравнения
- Как передать в функцию `sort` произвольный код?

- Скомпилированный (бинарный) код программы – тоже числа, и они хранятся где-то в памяти

- Скомпилированный (бинарный) код программы – тоже числа, и они хранятся где-то в памяти
 - Гарвардская архитектура: данные и код хранятся в разных типах памяти

- Скомпилированный (бинарный) код программы – тоже числа, и они хранятся где-то в памяти
 - Гарвардская архитектура: данные и код хранятся в разных типах памяти
 - Архитектура фон Неймана: данные и код хранятся в одной и той же памяти

- Скомпилированный (бинарный) код программы – тоже числа, и они хранятся где-то в памяти
 - Гарвардская архитектура: данные и код хранятся в разных типах памяти
 - Архитектура фон Неймана: данные и код хранятся в одной и той же памяти
- Так как это тоже память, у неё тоже есть адрес, и на неё тоже можно взять указатель

Указатели на функции

```
int add3(int x) {  
    return x + 3;  
}  
  
int main() {  
    // Объявление указателя на функцию  
    // Похоже на объявление функции, но со звёздочкой  
    int (*func)(int);  
  
    // Эти две строчки эквивалентны  
    func = &add3;  
    func = add3;  
  
    // Указатель на функцию можно вызвать так же, как  
    // обычную функцию  
    int x = func(5);  
}
```

- На любую функцию можно взять указатель (но только на саму функцию, а не на произвольное место в коде)

- На любую функцию можно взять указатель (но только на саму функцию, а не на произвольное место в коде)
- Указатели типизированные – знают, какие у функции аргументы и возвращаемое значение

- На любую функцию можно взять указатель (но только на саму функцию, а не на произвольное место в коде)
- Указатели типизированные – знают, какие у функции аргументы и возвращаемое значение
- Для указателей на функцию нет арифметики указателей

Сравнение объектов (compare)

```
void sort(void *array, size_t count, size_t elem_size,
          int (*compare)(void*, void*)) {
    ...
}

int compare_int(void *p1, void *p2) {
    return *(int *)p1 - *(int *)p2;
}

int compare_product(void *p1, void *p2) {
    return ((product_t*)p1)->weight - ((product_t*)p2)->weight;
}

int array[100];
sort(array, 100, sizeof(int), compare_int);
```

- **Стек (stack)**
 - Локальные переменные, параметры функций, адрес возврата
 - Код для выделения и освобождения генерирует компилятор
 - Выделяется при входе в функцию, освобождается при выходе

- **Стек (stack)**
 - Локальные переменные, параметры функций, адрес возврата
 - Код для выделения и освобождения генерирует компилятор
 - Выделяется при входе в функцию, освобождается при выходе
- **Глобальная (статическая, static) память**
 - Глобальные и статические переменные
 - Код для выделения и освобождения генерирует компилятор
 - Выделяется при старте программы, освобождается при её окончании

- **Стек (stack)**
 - Локальные переменные, параметры функций, адрес возврата
 - Код для выделения и освобождения генерирует компилятор
 - Выделяется при входе в функцию, освобождается при выходе
- **Глобальная (статическая, static) память**
 - Глобальные и статические переменные
 - Код для выделения и освобождения генерирует компилятор
 - Выделяется при старте программы, освобождается при её окончании
- **Динамическая память (куча, heap)**
 - Полностью управляется программистом

Адресное пространство

- Точное устройство адресного пространства зависит от системы
- Упрощённый современный линукс:

0x0	0x400000	0x40xxxx	0x5xxxxxxxxxxx	0x7ffxxxxxxxxx
Пусто	Код программы	Глобальные переменные	Куча (растёт туда ->)	Стек (растёт <- туда)

- Выделяется операционной системой при запуске программы (напр. 8 Мб)

- Выделяется операционной системой при запуске программы (напр. 8 Мб)
- Растёт “вниз” (в сторону меньших адресов)

- Выделяется операционной системой при запуске программы (напр. 8 Мб)
- Растёт “вниз” (в сторону меньших адресов)
- Используется в основном для вызова функций

```
int sum(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int c = 1, d = 2;  
    int e = sum(c, d);  
    return 0;  
}
```

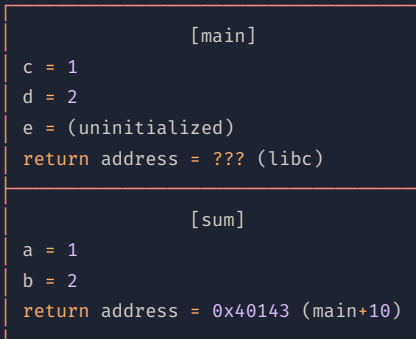
```
int sum(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int c = 1, d = 2;  
    int e = sum(c, d);  
    return 0;  
}
```

- При вызове функции нужно
 - Выделить место под её аргументы
 - Выделить место под её локальные переменные
 - Выделить место под её возвращаемое значение
 - Куда-то положить адрес возврата (куда должен вернуться **return**)

```
int sum(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int c = 1, d = 2;  
    int e = sum(c, d);  
    return 0;  
}
```

- При вызове функции нужно
 - Выделить место под её аргументы
 - Выделить место под её локальные переменные
 - Выделить место под её возвращаемое значение
 - Куда-то положить адрес возврата (куда должен вернуться **return**)
- $\implies$ *стек фрейм (stack frame)*

- Стек состоит из *фреймов* для всех вызванных в текущий момент функций (включая функции, вызвавшие другие функции)



- Размер стек фрейма легко вычислить компилятору – он знает количество аргументов и локальных переменных

- Размер стек фрейма легко вычислить компилятору – он знает количество аргументов и локальных переменных
- Адрес текущей вершины стека хранится в специальном регистре SP (stack pointer)

- Размер стек фрейма легко вычислить компилятору – он знает количество аргументов и локальных переменных
- Адрес текущей вершины стека хранится в специальном регистре SP (stack pointer)
- Выделить N байт на стеке – то же самое, что сделать $SP -= N$, т.е. очень быстро

- Размер стек фрейма легко вычислить компилятору – он знает количество аргументов и локальных переменных
- Адрес текущей вершины стека хранится в специальном регистре SP (stack pointer)
- Выделить N байт на стеке – то же самое, что сделать $SP -= N$, т.е. очень быстро
- $\implies$ создание локальных переменных – крайне быстрая операция

- Читать/писать в стек быстро, но это всё равно работа с памятью

- Читать/писать в стек быстро, но это всё равно работа с памятью
- Быстрее – передавать параметры и возвращаемое значение через регистры процессора (если они влезут)

- Читать/писать в стек быстро, но это всё равно работа с памятью
- Быстрее – передавать параметры и возвращаемое значение через регистры процессора (если они влезут)
- Договорённость о том, какие регистры под это использовать, является частью т.н. *calling convention*

```
int factorial(int n) {  
    if (n == 0) {  
        return 1;  
    }  
    return n * factorial(n - 1);  
}
```

```
int factorial(int n) {  
    if (n == 0) {  
        return 1;  
    }  
    return n * factorial(n - 1);  
}
```

- Каждый рекурсивный вызов выделяет свой фрейм

```
int factorial(int n) {  
    if (n == 0) {  
        return 1;  
    }  
    return n * factorial(n - 1);  
}
```

- Каждый рекурсивный вызов выделяет свой фрейм
- Если глубина рекурсии слишком большая, память стека закончится и программа упадёт с ошибкой *stack overflow*

```
int factorial(int n) {  
    if (n == 0) {  
        return 1;  
    }  
    return n * factorial(n - 1);  
}
```

- Каждый рекурсивный вызов выделяет свой фрейм
- Если глубина рекурсии слишком большая, память стека закончится и программа упадёт с ошибкой *stack overflow*
- Почти всегда либо глубина ограничена свойствами структуры данных (напр. $O(\log n)$ у деревьев), либо алгоритм можно переписать в виде цикла вместо рекурсии

Факториал без рекурсии

```
int factorial(int n) {  
    int result = 1;  
    while (n > 0) {  
        result *= n;  
        --n;  
    }  
    return result;  
}
```

Хвостовая рекурсия

```
int factorial(int n, int result) {  
    if (n == 0) {  
        return result;  
    }  
    return factorial(n - 1, result * n);  
}
```

Хвостовая рекурсия

```
int factorial(int n, int result) {  
    if (n == 0) {  
        return result;  
    }  
    return factorial(n - 1, result * n);  
}
```

- Если рекурсивный вызов это **самая последняя операция** перед **return**, компилятор может переиспользовать фрейм текущей функции и не выделять новый

Хвостовая рекурсия

```
int factorial(int n, int result) {  
    if (n == 0) {  
        return result;  
    }  
    return factorial(n - 1, result * n);  
}
```

- Если рекурсивный вызов это **самая последняя операция** перед **return**, компилятор может переиспользовать фрейм текущей функции и не выделять новый
- Эта оптимизация называется *хвостовой рекурсией (tail call optimization)*

- Переменные, объявленные снаружи любой функции, называются глобальными

Глобальные переменные

- Переменные, объявленные снаружи любой функции, называются глобальными
- Они находятся в отдельной области памяти, и живут в течении всей жизни программы

Глобальные переменные

- Переменные, объявленные снаружи любой функции, называются глобальными
- Они находятся в отдельной области памяти, и живут в течении всей жизни программы

```
int random_seed = 0;

void set_seed(int seed) {
    random_seed = seed;
}

int random() {
    random_seed = (random_seed * 13 + 113) % 43;
    return random_seed;
}
```

- Объявить одну и ту же глобальную переменную в нескольких файлах – ошибка multiple definition

Глобальные переменные

- Объявить одну и ту же глобальную переменную в нескольких файлах – ошибка multiple definition
- Можно исправить, пометив одно из объявлений как **extern**

```
// a.cpp
```

```
// Аналог определения  
// (definition) функции  
int global = 0;
```

```
// b.cpp
```

```
// Аналог объявления  
// (declaration) функции  
extern int global;
```

Глобальные переменные + static

- Глобальная переменная, объявленная со словом **static**, видна **только в этом файле** и не приводит к ошибке линковки

```
// a.cpp
```

```
static int global = 0;
```

```
// b.cpp
```

```
// Ошибки нет: в каждом файле  
// своя переменная global  
static int global = 0;
```

Статические переменные

- Переменные внутри функции, объявленные со словом `static`
- Живут вместе с глобальными переменными, но доступны только внутри этой функции

```
void foo() {  
    // Инициализируется один раз  
    // Значение сохраняется даже после выхода из функции  
    static int call_count = 0;  
  
    printf("I was called %i times\n", call_count);  
    ++call_count;  
}
```

- При компиляции компилятор может легко посчитать, сколько памяти нужно под все глобальные/статические переменные в этом файле

- При компиляции компилятор может легко посчитать, сколько памяти нужно под все глобальные/статические переменные в этом файле
- При линковке линкофщик может легко просуммировать необходимую память под все глобальные/статические переменные всех файлов

- При компиляции компилятор может легко посчитать, сколько памяти нужно под все глобальные/статические переменные в этом файле
- При линковке линкофщик может легко просуммировать необходимую память под все глобальные/статические переменные всех файлов
- Эта память выделится в начале работы программы и освободится при её завершении

- Потенциальный конфликт имён, если два программиста завели в разных частях программы одинаковые глобальные переменные

Глобальные переменные: недостатки

- Потенциальный конфликт имён, если два программиста завели в разных частях программы одинаковые глобальные переменные
- Трудно анализировать, какие части программы что используют и на что могут повлиять

Глобальные переменные: недостатки

- Потенциальный конфликт имён, если два программиста завели в разных частях программы одинаковые глобальные переменные
- Трудно анализировать, какие части программы что используют и на что могут повлиять
- Порядок инициализации глобальных переменных в рамках всей программы **не определён** (только в рамках одного файла)

Глобальные переменные: недостатки

- Потенциальный конфликт имён, если два программиста завели в разных частях программы одинаковые глобальные переменные
- Трудно анализировать, какие части программы что используют и на что могут повлиять
- Порядок инициализации глобальных переменных в рамках всей программы **не определён** (только в рамках одного файла)
- Считаются плохим тоном/антипаттерном, рекомендуется их не использовать без крайней необходимости